

SELECTORS REDUX

Cours Theorique

Session 16 : Manipulation des Selectors

Organiser l'accès au state Redux

Module M203 - Développement Front-End

ISTA - 2eme Annee DEVOWFS

Annee 2024/2025

Objectifs de la seance (2h30) :

- Comprendre le rôle des **selectors** dans Redux
- Créer des **selectors simples** pour extraire le state
- Créer des **selectors avec logique** (filtrage, tri, calculs)
- Créer des **selectors avec parametres**
- Organiser les selectors dans un projet

Duree : 2h30

Prerequis : S14 (Redux), S15 (RTK, useSelector)

Prochaine session : S17 - Formulaires & Listes

Table des matières

1	Introduction aux Selectors	2
1.1	Rappel : useSelector	2
1.2	Problemes de l'approche inline	2
1.3	La Solution : Selectors Dedics	2
2	Selectors Simples	3
2.1	Creation d'un Selector Simple	3
2.2	Utilisation dans un Composant	3
3	Selectors avec Logique	4
3.1	Filtrage de Donnees	4
3.2	Tri de Donnees	4
3.3	Calculs et Aggregations	5
3.4	Combinaison de Filtres	5
4	Selectors avec Parametres	6
4.1	Le Probleme	6
4.2	Solution : Selector avec Plusieurs Arguments	6
4.3	Utilisation dans un Composant	6
5	Organisation des Selectors	7
5.1	Structure Recommandee	7
5.2	Convention de Nommage	7
5.3	Exemple Complet d'un Fichier Selectors	7

1 Introduction aux Selectors

1.1 Rappel : useSelector

En S14 et S15, vous avez utilise `useSelector` pour lire le state :

```
1 // Lire directement dans le composant
2 const items = useSelector((state) => state.panier.items);
3 const count = useSelector((state) => state.panier.items.length);
```

Cette approche fonctionne, mais pose des problemes quand l'application grandit.

1.2 Problemes de l'approche inline

Attention

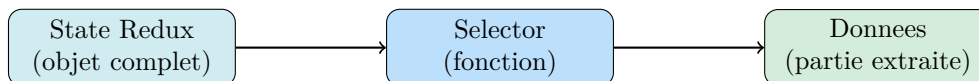
Problemes avec les selectors inline :

1. **Duplication** : Meme code copie dans plusieurs composants
2. **Maintenance** : Si la structure du state change, il faut modifier partout
3. **Testabilite** : Impossible de tester la logique d'extraction isolément
4. **Lisibilite** : Logique complexe melangee avec le rendu

1.3 La Solution : Selectors Dedies

Definition

Un **selector** est une fonction qui prend le **state** complet en parametre et retourne une partie ou une transformation de ce state.



2 Selectors Simples

2.1 Creation d'un Selector Simple

Un selector simple extrait une partie du state sans transformation.

Syntaxe

```
1 // src/features/panier/panierSelectors.js
2
3 // Selector simple : extraire le tableau items
4 export const selectPanierItems = (state) => state.panier.items;
5
6 // Selector simple : extraire le nombre d'items
7 export const selectPanierCount = (state) => state.panier.items.
  length;
8
9 // Selector simple : verifier si le panier est vide
10 export const selectPanierIsEmpty = (state) => state.panier.items.
  length === 0;
```

2.2 Utilisation dans un Composant

```
1 // src/components/Panier.jsx
2
3 import { useSelector } from 'react-redux';
4 import { selectPanierItems, selectPanierCount } from '../features/panier/
  panierSelectors';
5
6 function Panier() {
7   // Utiliser les selectors importes
8   const items = useSelector(selectPanierItems);
9   const count = useSelector(selectPanierCount);
10
11   return (
12     <div>
13       <h2>Panier ({count} articles)</h2>
14       <ul>
15         {items.map((item, index) => (
16           <li key={index}>{item}</li>
17         ))}
18       </ul>
19     </div>
20   );
21 }
```

Bonne Pratique

Avantages :

- Le composant ne connaît pas la structure interne du state
- Si `state.panier` devient `state.cart`, on modifie uniquement le selector
- Le même selector peut être réutilisé dans plusieurs composants

3 Selectors avec Logique

3.1 Filtrage de Donnees

Les selectors peuvent contenir de la logique métier comme le filtrage.

Syntaxe

```
1 // src/features/products/productsSelectors.js
2
3 // Selector de base
4 export const selectProducts = (state) => state.products.items;
5
6 // Selector avec filtrage : produits disponibles (stock > 0)
7 export const selectAvailableProducts = (state) =>
8   state.products.items.filter(product => product.stock > 0);
9
10 // Selector avec filtrage : produits d'une categorie
11 export const selectTechProducts = (state) =>
12   state.products.items.filter(product => product.category === 'tech
13   ');
14
15 // Selector avec filtrage : produits chers (prix > 100)
16 export const selectExpensiveProducts = (state) =>
17   state.products.items.filter(product => product.price > 100);
```

3.2 Tri de Donnees

```
1 // Selector avec tri : produits tries par prix (croissant)
2 export const selectProductsSortedByPrice = (state) =>
3   [...state.products.items].sort((a, b) => a.price - b.price);
4
5 // Selector avec tri : produits tries par nom (alphabetique)
6 export const selectProductsSortedByName = (state) =>
7   [...state.products.items].sort((a, b) => a.name.localeCompare(b.name));
8
9 // Selector avec tri : produits tries par stock (decroissant)
10 export const selectProductsSortedByStock = (state) =>
11   [...state.products.items].sort((a, b) => b.stock - a.stock);
```

Attention

Important : Utilisez `[...array]` pour créer une copie avant de trier. La méthode `sort()` modifie le tableau original, ce qui violerait l'immuabilité du state !

3.3 Calculs et Aggregations

Syntaxe

```
1 // src/features/panier/panierSelectors.js
2
3 // Selector avec calcul : prix total du panier
4 export const selectPanierTotal = (state) => {
5   const panierItems = state.panier.items;
6   const products = state.products.items;
7
8   return panierItems.reduce((total, item) => {
9     const product = products.find(p => p.id === item.productId);
10    return total + (product?.price || 0) * item.quantity;
11  }, 0);
12 };
13
14 // Selector avec calcul : nombre total d'articles
15 export const selectTotalQuantity = (state) =>
16   state.panier.items.reduce((total, item) => total + item.quantity,
17     0);
18
19 // Selector avec calcul : valeur totale du stock
20 export const selectTotalStockValue = (state) =>
21   state.products.items.reduce((total, p) => total + p.price * p.
22     stock, 0);
```

3.4 Combinaison de Filtres

```
1 // Selector avec filtres combines
2 export const selectFilteredProducts = (state) => {
3   const products = state.products.items;
4   const { category, minPrice, maxPrice } = state.filters;
5
6   return products.filter(product => {
7     // Filtre par categorie (si pas 'all')
8     const matchCategory = category === 'all' || product.category ===
9       category;
10
11     // Filtre par prix minimum
12     const matchMinPrice = product.price >= minPrice;
13
14     // Filtre par prix maximum
15     const matchMaxPrice = product.price <= maxPrice;
16
17     // Le produit doit passer TOUS les filtres
18     return matchCategory && matchMinPrice && matchMaxPrice;
19   });
20 };
```

4 Selectors avec Parametres

4.1 Le Probleme

Parfois, on veut un selector qui accepte un parametre dynamique (ex : ID d'un produit).

4.2 Solution : Selector avec Plusieurs Arguments

Syntaxe

```
1 // Selector qui accepte state ET un parametre
2 export const selectProductById = (state, productId) =>
3   state.products.items.find(product => product.id === productId);
4
5 // Selector qui accepte state ET plusieurs parametres
6 export const selectProductsByPriceRange = (state, minPrice, maxPrice
7   ) =>
8   state.products.items.filter(p => p.price >= minPrice && p.price <=
9     maxPrice);
10
11 // Selector de recherche par nom
12 export const selectProductsBySearch = (state, searchTerm) =>
13   state.products.items.filter(p =>
14     p.name.toLowerCase().includes(searchTerm.toLowerCase())
15   );
```

4.3 Utilisation dans un Composant

```
1 // Exemple 1 : Trouver un produit par ID
2 function ProductDetail({ productId }) {
3   const product = useSelector((state) => selectProductById(state,
4     productId));
5
6   return <div>{product?.name} - {product?.price} DH</div>;
7 }
8
9 // Exemple 2 : Recherche dynamique
10 function ProductSearch() {
11   const [search, setSearch] = useState('');
12
13   const products = useSelector((state) =>
14     selectProductsBySearch(state, search)
15   );
16
17   return (
18     <div>
19       <input
20         value={search}
21         onChange={(e) => setSearch(e.target.value)}
22         placeholder="Rechercher..."
23       />
24       <ul>
25         {products.map(p => <li key={p.id}>{p.name}</li>)}
26       </ul>
27     </div>
28   );
```

```
28 }
```

5 Organisation des Selectors

5.1 Structure Recommandee

```
src/
  app/
    store.js
  features/
    products/
      productsSlice.js      # State + Reducers
      productsSelectors.js  # Selectors dedies
    panier/
      panierSlice.js
      panierSelectors.js
    filters/
      filtersSlice.js
      filtersSelectors.js
  components/
    ...
```

5.2 Convention de Nommage

Type	Convention	Exemple
Selector simple	select + Nom	selectProducts
Selector booleen	select + Is/Has + Nom	selectIsLoading
Selector avec filtre	select + Nom + Filtre	selectAvailableProducts
Selector avec calcul	select + Total/Count + Nom	selectTotalPrice
Selector avec parametre	select + Nom + By	selectProductById

5.3 Exemple Complet d'un Fichier Selectors

```
1 // src/features/products/productsSelectors.js
2
3 // ===== SELECTORS SIMPLES =====
4 export const selectProducts = (state) => state.products.items;
5 export const selectProductsLoading = (state) => state.products.loading;
6 export const selectProductsError = (state) => state.products.error;
7 export const selectProductsCount = (state) => state.products.items.length;
8
9 // ===== SELECTORS AVEC LOGIQUE =====
10 export const selectAvailableProducts = (state) =>
11   state.products.items.filter(p => p.stock > 0);
12
13 export const selectProductsSortedByPrice = (state) =>
14   [...state.products.items].sort((a, b) => a.price - b.price);
15
16 export const selectTotalStockValue = (state) =>
17   state.products.items.reduce((total, p) => total + p.price * p.stock, 0);
18
19 // ===== SELECTORS AVEC PARAMETRES =====
20 export const selectProductById = (state, id) =>
```



```
21   state.products.items.find(p => p.id === id);
22
23   export const selectProductsByCategory = (state, category) =>
24     state.products.items.filter(p => p.category === category);
25
26   export const selectProductsBySearch = (state, term) =>
27     state.products.items.filter(p =>
28       p.name.toLowerCase().includes(term.toLowerCase())
29     );
```